

UNIT-III- ASP.NET Page

ASP.NET life cycle specifies, how:

- ASP.NET processes pages to produce dynamic output
- The application and its pages are instantiated and processed
- ASP.NET compiles the pages dynamically

The ASP.NET life cycle could be divided into two groups:

- Application Life Cycle
- Page Life Cycle

ASP.NET Application Life Cycle

The application life cycle has the following stages:

- User makes a request for accessing application resource, a page. Browser sends this request to the web server.
- A unified pipeline receives the first request and the following events take place:
- An object of the class `ApplicationManager` is created.
- An object of the class `HostingEnvironment` is created to provide information regarding the resources.
- Top level items in the application are compiled.
- Response objects are created. The application objects such as `HttpContext`, `HttpRequest` and `HttpResponse` are created and initialized.
- An instance of the `HttpApplication` object is created and assigned to the request.
- The request is processed by the `HttpApplication` class. Different events are raised by this class for processing the request.

ASP.NET Page Life Cycle

When a page is requested, it is loaded into the server memory, processed, and sent to the browser. Then it is unloaded from the memory. At each of these steps, methods and events are available, which could be overridden according to the need of the application. In other words, you can write your own code to override the default code.

The `Page` class creates a hierarchical tree of all the controls on the page. All the components on the page, except the directives, are part of this control tree. You can see the control tree by adding `trace= "true"` to the page directive. We will cover page directives and tracing under 'directives' and 'event handling'.

The page life cycle phases are:

- Initialization
- Instantiation of the controls on the page
- Restoration and maintenance of the state
- Execution of the event handler codes

Page rendering

Understanding the page cycle helps in writing codes for making some specific thing happen at any stage of the page life cycle. It also helps in writing custom controls and initializing them at right time, populate their properties with view-state data and run control behavior code.

Following are the different stages of an ASP.NET page:

Page request - When ASP.NET gets a page request, it decides whether to parse and compile the page, or there would be a cached version of the page; accordingly the response is sent.

Starting of page life cycle - At this stage, the Request and Response objects are set. If the request is an old request or post back, the IsPostBack property of the page is set to true. The UICulture property of the page is also set.

Page initialization - At this stage, the controls on the page are assigned unique ID by setting the UniqueID property and the themes are applied. For a new request, postback data is loaded and the control properties are restored to the view-state values.

Page load - At this stage, control properties are set using the view state and control state values.

Validation - Validate method of the validation control is called and on its successful execution, the IsValid property of the page is set to true.

Postback event handling - If the request is a postback (old request), the related event handler is invoked.

Page rendering - At this stage, view state for the page and all controls are saved. The page calls the Render method for each control and the output of rendering is written to the OutputStream class of the Response property of page.

Unload - The rendered page is sent to the client and page properties, such as Response and Request, are unloaded and all cleanup done.

ASP.NET Page Life Cycle Events

At each stage of the page life cycle, the page raises some events, which could be coded. An event handler is basically a function or subroutine, bound to the event, using declarative attributes such as Onclick or handle.

Following are the page life cycle events:

PreInit - PreInit is the first event in page life cycle. It checks the IsPostBack property and determines whether the page is a postback. It sets the themes and master pages, creates dynamic controls, and gets and sets profile property values. This event can be handled by overloading the OnPreInit method or creating a Page_PreInit handler.

Init - Init event initializes the control property and the control tree is built. This event can be handled by overloading the OnInit method or creating a Page_Init handler.

InitComplete - InitComplete event allows tracking of view state. All the controls turn on view-state tracking.

LoadViewState - LoadViewState event allows loading view state information into the controls.

LoadPostData - During this phase, the contents of all the input fields are defined with the <form> tag are processed.

PreLoad - PreLoad occurs before the post back data is loaded in the controls. This event can be handled by overloading the OnPreLoad method or creating a Page_PreLoad handler.

Load - The Load event is raised for the page first and then recursively for all child controls. The controls in the control tree are created. This event can be handled by overloading the OnLoad method or creating a Page_Load handler.

LoadComplete - The loading process is completed, control event handlers are run, and page validation takes place. This event can be handled by overloading the OnLoadComplete method or creating a Page_LoadComplete handler

PreRender - The PreRender event occurs just before the output is rendered. By handling this event, pages and controls can perform any updates before the output is rendered.

PreRenderComplete - As the PreRender event is recursively fired for all child controls, this event ensures the completion of the pre-rendering phase.

SaveStateComplete - State of control on the page is saved. Personalization, control state and view state information is saved. The HTML markup is generated. This stage can be handled by overriding the Render method or creating a Page_Render handler.

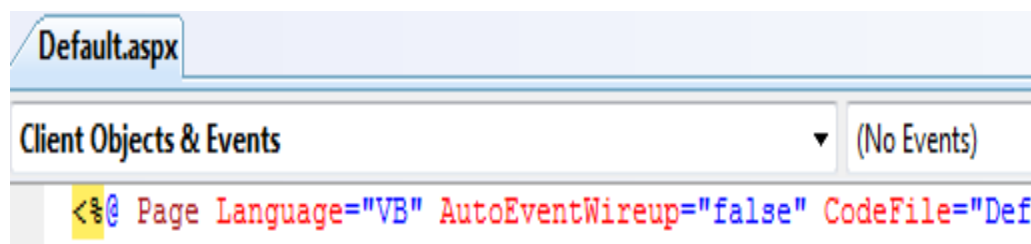
UnLoad - The UnLoad phase is the last phase of the page life cycle. It raises the UnLoad event for all controls recursively and lastly for the page itself. Final cleanup is done and all resources and references, such as database connections, are freed. This event can be handled by modifying the OnUnLoad method or creating a Page_UnLoad handler.

Page Directives in Asp.Net

Directives

The directives are instructions that specify optional settings in Asp.Net, but they are not rendered as part of the HTML page return to the client browser. These instructions include registering a custom control, page language etc. It describes how the .aspx pages (web forms) or .ascx pages (user controls) are processed by the .Net framework.

Page directive



The most commonly used directive is the @ Page directive and it can be used only in Web Forms. Page directive allows you to specify many configuration options for the page. By default, Visual Studio creates a page directive as shown below:



You can include only one @ Page directive in your .aspx file. Also you should specify one language in the Language attribute. This can be any .NET Framework-supported language, including VB.Net, C#, or JScript.

AutoEventWireup controlled the automatic binding of page events based on the method naming convention. The default is true, which performs the automatic lookup and binding. If it is set to False then you should create methods with any name and bind them to page events explicitly

CodeFile specifies a path to the referenced code-behind file for the page. Inherits defines the name of the class from which to inherit. This can be any class derived from the Page class

Different types of directives in Asp.Net

@ Assembly

The @Assembly Directive attaches assemblies to the page or an ASP.NET user control thereby all the assembly classes and interfaces are available to the class. This directive supports the two attributes Name and src. The Name attribute defines the assembly name and the src attribute defines the source of the assembly.

@ Control

Defines control-specific attributes used by the ASP.NET page parser and compiler and can be included only in .ascx files (user controls).

@ Implements

The @Implements Directive gets the ASP.NET pages to implement .Net framework interfaces. This directive only supports a single attribute interface.

@ Import

The Import directive imports a namespace into a web page, user control page of application. If the Import directive is specified in the global.asax file, then it is applied to the entire application. If it is in a page of user control page, then it is applied to that page or control.

@ Master

Identifies a page as a master page and defines attributes used by the ASP.NET page parser and compiler and can be included only in .master files.

@ MasterType

Defines the class or virtual path used to type the Master property of a page.

@ OutputCache

The OutputCache directive controls the output caching policies of a web page or a user control.

@ Page

The @Page directive enables you to specify attributes and values for an Asp.Net Page to be used when the page is parsed and compiled. Every .aspx files should include this @Page directive to execute. There are many attributes belong to this directive.

@ PreviousPageType

Creates a strongly typed reference to the source page from the target of a cross-page posting.

@ Reference

Links a page, user control, or COM control to the current page or user control declaratively.

@ Register

Associates aliases with namespaces and classes, which allow user controls and custom server controls to be rendered when included in a requested page or user control.

Application Directory Structure in ASP.NET

The asp.net application folder contains a list of specified folders that you can use of specific types of files or content in each folder. The root folder structure is as follows:

- BIN
- App_Code
- App_GlobalResources
- App_LocalResources
- App_WebReferences
- App_Data
- App_Browsers
- App_Themes

Bin Directory

It contains all the precompiled .Net assemblies like DLLs that the application uses.

App_Code Directory

It contains source code files like .cs or .vb that are dynamically compiled for use in your application. These source code files are usually separate components or a data access library.

App_GlobalResources Directory

It contains and stores global resources that are accessible to every page.

App_LocalResources Directory

It serves the same purpose as app_globalresources, except these resources are accessible for their dedicated page only.

App_WebReferences Directory

It stores reference to web services that the web application uses.

App_Data Directory

It is reserved for data storage and also mdf files, xml file and so on.

App_Browsers Directory

It contains browser definitions stored in xml files. These xml files define the capabilities of client side browsers for different rendering actions.

App_Themes Directory

It contains collection of files like .skin and .css files that used to application look and feel appearance.

ASP.NET defines a few directories with special meanings.

Directory	Description
Bin	Contains all the precompiled .NET assemblies usually DLLs
App_Code	Contains source code files that are dynamically compiled for use
App_GlobalResources	stores global resources
App_LocalResources	resources are accessible for their dedicated page only
App_WebReferences	Stores references to web services that the web application uses. This includes WSDL files and discovery documents.
App_Data	reserved for data storage
App_Browsers	browser definitions stored in XML files
App_Themes	themes used by the web application

ASP.NET Compilation

ASP.NET is nothing like the legacy ASP with which many developers are familiar. You develop ASP pages by using VBScript or JScript, and they are interpreted, meaning that they are executed just as they are written, directly from the page. ASP.NET is entirely different in that ASP.NET pages are compiled before they are executed.

When you write ASP.NET code, you do so in human-readable text. Before ASP.NET can run your code, it has to convert it into something that the computer can understand and execute. The process of converting code from what a programmer types into what a computer can actually execute is called compilation.

The Web Application Compilation Model

The web application compilation model is the same model provided in ASP.NET 1.0 and 1.1. When you use this model, you use the Build menu in Visual Web Developer to compile your application into a single DLL file that is copied to a bin folder in the root of your application. When the first request comes into your application, the DLL from the bin folder is copied to the Temporary ASP.NET Files folder, where it is then recompiled into code that the operating system can execute in a process known as just-in-time (JIT) compilation. The JIT compilation causes a delay of several seconds on the first request of the application.

The Precompilation Model

- The precompilation model allows you to compile your ASP.NET application into one or more DLLs that can then be copied to the web server in place of any code.
- ASP.NET Precompilation
- ASP.NET can precompile a Web site before it is made available to users. This provides many advantages, which include faster response time, error checking, source-code protection, and efficient deployment.

Precompiling an ASP.NET Web site provides the following advantages:

- Faster response time for users, because pages and code files do not have to be compiled the first time that they are requested. This is especially useful on large sites that are frequently updated.
- A means to identify compile-time bugs before users see a site.
- The ability to create a compiled version of the site that can be deployed to a production server without source code.

Precompilation for Deployment

Another use for precompiling a site is to produce an executable version of the site that can be deployed to a production server. Precompiling for deployment creates output in the form of a layout. The layout contains assemblies, configuration information, information about the site's folders, and static files (such as HTML files and graphics). After compiling the site, you can deploy the layout to a production server by using tools such as the Windows XCopy command, FTP, Windows installation, and so on. After the layout is deployed, it functions as the site, and ASP.NET fulfills requests for pages from the assemblies in the layout. Precompiling a site for deployment provides a measure of protection for source code and other intellectual property. For more information about how the compiler works with files during compilation for deployment

Precompiling for Deployment Only

When you precompile for deployment only, the compiler produces assemblies from almost all ASP.NET source files that are ordinarily compiled at run time. This includes program code in pages, .cs and .vb class files, other code files, and resource files. The compiler removes all source and markup from the output. In the resulting layout, compiled files are generated for each .aspx file (with the extension .compiled) that contains pointers to the appropriate assembly for that page. To change the Web site, which includes the layout of pages, you must change the original files, recompile the site, and redeploy the layout. The only exception is the site configuration. You can make changes to the Web.config file on the production server without having to recompile the site. This option provides the greatest degree of protection for ASP.NET Web pages and the best performance at startup.

Precompiling for Deployment and Update

When you precompile for deployment and update, the compiler produces assemblies from all source code (except page code in single-file pages) and from other files that ordinarily produce assemblies, such as resource files. The compiler converts .aspx files into single files that use the compiled code-behind model and copies them to the layout.

This option enables you to make limited changes to the ASP.NET Web pages in the site after you compile them. For example, you can change the arrangement of controls, colors, fonts, and other appearance aspects of pages. You can also add controls, as long as they do not require event handlers or other code.